

Module 4 Journal

1) Simple Analogy

Picture our game as a food truck (server) and players as customers arriving by different roads (clients: web, mobile, desktop). The truck owns the recipes and cooking (rules, data, security). Customers order from a simple, universal menu (REST over HTTP/JSON). Because the menu is standard and the truck location is known, any customer can be served. That's the client-server split: clear responsibilities so each side can evolve independently without breaking the other.

2) Why Client-Server Satisfies Requirements

This architecture lets each client ship UI/UX changes independently while the server remains the authoritative source of truth for rules and data, ensuring consistency across platforms.

Separating concerns lets us change visuals and input devices on clients while keeping the rules and data on the server. Portability comes from HTTP/JSON: browsers, Android, iOS, desktop, and later Xbox/PS4 can all talk to the same API. Scaling is straightforward because requests are stateless—multiple server instances behind a load balancer can respond. Centralizing business logic improves resilience and security (uniform auth, validation, rate limiting, and monitoring). This architecture therefore meets the requirement that the game run on multiple operating platforms while remaining consistent and secure.

3) Server Side with REST

Contract first: Define resources with OpenAPI: /users, /games, /sessions, /scores.

HTTP verbs: GET reads, POST creates, PUT/PATCH updates, DELETE removes.

Representations & versioning: JSON with stable schemas (e.g., Accept: application/vnd.tgr.v1+json).

Statelessness: Each request carries identity and context so any instance can process it.

Hygiene: Pagination and filtering for large lists; clear error models; idempotency keys for sensitive writes; CORS for approved origins; structured logs and metrics for observability.

3a) Security (Principal, Authenticator, Authorizer, Annotation, OS Integration)

Example: POST /matches is protected by @RolesAllowed({"USER","ADMIN"}), and each resource receives @Auth UserPrincipal principal. The authorizer enforces principal.tenantId == body.tenantId before writes.

Principal: A UserPrincipal (id, username, roles, tenantId) represents the authenticated caller that resources trust.

Authenticator: Verifies credentials (JWT signature/expiry or Basic with hashed password using BCrypt/Argon2). On success, it hydrates the UserPrincipal.

Authorizer: Enforces RBAC and attribute checks (e.g., matching tenantId). Resources ask, “Can this principal do X on Y?”

Annotations: Inject identity with @Auth UserPrincipal in resource methods and guard actions with @RolesAllowed or custom scope annotations.

OS/Enterprise integration: Hook into SSO (OIDC/SAML via AD/LDAP). Store secrets with OS protections; forward tamper-evident audit logs to syslog.

Protection: TLS/HSTS for transport, encryption at rest for sensitive columns, multi-tenant scoping on every query, and audit trails for all mutations.

Why server-side separation helps: Clients are easy to inspect/modify; putting checks on the server prevents cheating and data tampering while enabling key rotation and revocation in one place.

4) Client Side for Multiple Environments

To support three clients (e.g., web, Android, iOS), developers should generate SDKs from the OpenAPI contract so models and endpoints stay in sync, centralize API utilities (auth token handling, retries, exponential backoff), and follow platform-appropriate UX. The website should be responsive and accessible; mobile apps should use native navigation and secure token storage (Keychain/Keystore). All clients handle common states (offline, 401, 429) gracefully, cache immutable data (e.g., leaderboards), and load heavy assets from a CDN. Internationalization (i18n) makes text and formats locale-aware, and WCAG guidance ensures accessibility.

5) Next Steps & Features

Adding users: Implement POST /users with validation for unique email/handle; manage schema changes with migrations (Flyway/Liquibase) and add indices for lookups. Provide a bulk endpoint (POST /users:batch) with rate limiting for tournaments.

Features: Matchmaking/lobbies (POST /matches with skill/region filters), leaderboards and seasons (GET /leaderboards?season= with server-calculated ranks), friends/invites (POST /invites) plus real-time presence via WebSockets/SSE, and telemetry to guide balancing. The server remains authoritative for all scoring and win conditions to preserve fairness.

More platforms (Xbox/PS4): Reuse the same REST API. Build thin console clients with each platform’s SDK, integrate platform auth (Xbox Live/PSN) with account linking to our IDs, design controller-first UI with TV-safe typography, and pass certification (TRCs/XRs). If cross-play is enabled, keep rules identical, enforce moderation policies, and ensure chat/content filters meet platform standards.

6) Interaction Contract, Checks, and Troubleshooting

Contract: Server owns state; clients render and request changes. Versioned endpoints preserve backward compatibility. Contract tests (Pact) and Postman collections run in CI to prevent breaking changes.

Checks: From web and mobile, play the same scenario and verify identical outcomes; throttle the network to confirm graceful retries; terminate a server instance mid-session to verify stateless failover.

Troubleshooting: Fix CORS preflight errors by aligning allowed origins/headers; mitigate 401 loops with token refresh and clock-skew tolerance; scale hot endpoints (leaderboards) via caching, pagination, and read replicas; use WebSockets for real-time presence with a polling fallback if needed.

Bottom line: Client-server with a well-designed REST API delivers portability, security, and independent evolution across today's web/mobile/desktop and tomorrow's consoles.

7) Conclusion

Bottom line: a RESTful, server-authoritative core plus versioned contracts gives us portability across web, mobile, and consoles, stronger security by centralizing checks and data, and safe, independent evolution of clients without breaking existing users.